Models overview

Claude is a family of state-of-the-art large language models developed by Anthropic. Compare the current lineup, find the model ID for every platform, and open each model's page for its full specs and resources.

Choosing a model | Pricing | Migration guide

Latest models

If you're unsure which model to use, start with Claude Opus 5.5 for most workloads. Use Claude Fable 5.2 for demanding reasoning and long-horizon agentic work, or when your evals on Claude Opus 5.5 at higher effort still fall short. All current models support text and image input, text output, multilingual capabilities, vision, and tool use.

	Claude Fable 5.2	Claude Opus 5.5	Claude Sonnet 5.5	Claude Haiku 5.5
	For demanding reasoning and long-horizon agentic work	For long-running agentic coding and knowledge work	The best combination of speed and intelligence	For high-volume, latency-sensitive tasks such as classification, extraction, and routing
Comparative latency	Moderate	Moderate	Fast	Fastest
Pricing	$8 / input MTok $40 / output MTok	$4 / input MTok $20 / output MTok	$2 / input MTok $10 / output MTok	From $0.10 / input MTok From $0.50 / output MTok

	 Claude Fable 5.2 For demanding reasoning and long-horizon agentic work	 Claude Opus 5.5 For long-running agentic coding and knowledge work	 Claude Sonnet 5.5 The best combination of speed and intelligence	 Claude Haiku 5.5 For high-volume, latency-sensitive tasks such as classification, extraction, and routing
Claude API ID	`claude-fable-5-2`	`claude-opus-5-5`	`claude-sonnet-5-5`	`claude-haiku-5-5`
Capabilities				
Thinking	Adaptive (always on)	Adaptive (always on)	Adaptive	Adaptive
Default effort	`high`	`medium`	`high`	`medium`
Context window	1M tokens	1M tokens	1M tokens	1M tokens
Max output	128K tokens	128K tokens	128K tokens	128K tokens
Reliable knowledge cutoff	Jun 2026	Jun 2026	Jun 2026	Jun 2026
Additional details				
Training data cutoff	Jun 2026	Jun 2026	Jun 2026	Jun 2026
Retirement	Not sooner than November 9, 2027	Not sooner than September 22, 2027	Not sooner than September 28, 2027	Not sooner than October 7, 2027
Model IDs				
Claude API alias	`claude-fable-5-2`	`claude-opus-5-5`	`claude-sonnet-5-5`	`claude-haiku-5-5`
Amazon Bedrock ID	`global.anthropic.claude-fable-5-2`	`anthropic.claude-opus-5-5`	`anthropic.claude-sonnet-5-5`	`anthropic.claude-haiku-5-5`

	Claude Fable 5.2 For demanding reasoning and long-horizon agentic work	Claude Opus 5.5 For long-running agentic coding and knowledge work	Claude Sonnet 5.5 The best combination of speed and intelligence	Claude Haiku 5.5 For high-volume, latency-sensitive tasks such as classification, extraction, and routing
Claude Models on Google Cloud (Google managed) ID	`claude-fable-5-2`	`claude-opus-5-5`	`claude-sonnet-5-5`	`claude-haiku-5-5`
Microsoft Foundry ID	`claude-fable-5-2`	`claude-opus-5-5`	`claude-sonnet-5-5`	`claude-haiku-5-5`
Claude Platform on AWS ID	`claude-fable-5-2`	`claude-opus-5-5`	`claude-sonnet-5-5`	`claude-haiku-5-5`
Claude Models on Google Cloud (Anthropic managed) ID	`claude-fable-5-2`	`claude-opus-5-5`	`claude-sonnet-5-5`	`claude-haiku-5-5`

Once you've picked a model, learn how to make your first API call. To understand how model IDs, aliases, and snapshots work, see Model IDs and versioning. For the reliable-knowledge and training-data cutoffs behind each model, see Anthropic's Transparency Hub.

All models

Older models might still be available, but where possible, you should default to the latest models. To move to a newer model, see the migration guides. For retirement dates, see model deprecations.

Model	Capabilities		Base tokens	
Name	Context window	Max output	Input	Output
Claude Fable 5.2	1M tokens	128K tokens	$8 / MTok	$40 / MTok

Model	Capabilities		Base tokens	
Name	Context window	Max output	Input	Output
Claude Opus 5.5	1M tokens	128K tokens	$4 / MTok	$20 / MTok
Claude Sonnet 5.5	1M tokens	128K tokens	$2 / MTok	$10 / MTok
Claude Haiku 5.5	1M tokens	128K tokens	From $0.10 / MTok	From $0.50 / MTok
Claude Mythos 5.2	1M tokens	128K tokens	$8 / MTok	$40 / MTok
Claude Fable 5.1	1M tokens	128K tokens	$10 / MTok	$50 / MTok
Claude Mythos 5.1	1M tokens	128K tokens	$10 / MTok	$50 / MTok
Claude Fable 5	1M tokens	128K tokens	$10 / MTok	$50 / MTok
Claude Mythos 5	1M tokens	128K tokens	$10 / MTok	$50 / MTok
Claude Opus 5	1M tokens	128K tokens	$5 / MTok	$25 / MTok
Claude Opus 4.8	1M tokens	128K tokens	$5 / MTok	$25 / MTok
Claude Opus 4.7	1M tokens	128K tokens	$5 / MTok	$25 / MTok
Claude Opus 4.6	1M tokens	128K tokens	$5 / MTok	$25 / MTok
Claude Opus 4.5	200K tokens	64K tokens	$5 / MTok	$25 / MTok
Claude Opus 4.1	200K tokens	32K tokens	$15 / MTok	$75 / MTok
Claude Opus 4	200K tokens	32K tokens	$15 / MTok	$75 / MTok

Model	Capabilities		Base tokens	
Name	Context window	Max output	Input	Output
Claude Sonnet 5	1M tokens	128K tokens	$2 / MTok	$10 / MTok
Claude Sonnet 4.6	1M tokens	128K tokens	$3 / MTok	$15 / MTok
Claude Sonnet 4.5	200K tokens	64K tokens	$3 / MTok	$15 / MTok
Claude Sonnet 4	200K tokens	64K tokens	$3 / MTok	$15 / MTok
Claude Haiku 4.5	200K tokens	64K tokens	$1 / MTok	$5 / MTok

Using the Models API

You can query model capabilities and token limits programmatically with the Models API. The response includes `max_input_tokens`, `max_tokens`, and a `capabilities` object for every available model.

Each model in the response also has a `line` field, which names the model line it belongs to. Claude Opus 4.5 and Claude Opus 4.6 both report `opus`. Use `line` to group models, for example, in a model picker. `line` is `null` when a model belongs to no line. Read `line` instead of inferring it from the model's `id`. Anthropic might add more lines, so don't treat the set of values as fixed.

Each model's `capabilities` object includes `thinking.types.disabled`, which reports whether the model accepts `thinking: {type: "disabled"}`, the setting that turns thinking off. `supported` is `false` when the model rejects `"disabled"` with a 400 error, and `true` on a model that doesn't support thinking. Even when `supported` is `true`, the API can still reject a `"disabled"` request for another reason. One such reason is an effort level that the model doesn't allow with thinking off.

Each model's `capabilities` object also includes `thinking.types.between_tools`, which reports whether the model accepts `thinking: {type: "between_tools"}`, the setting that turns off up-front thinking. `supported` is `false` when the model rejects `"between_tools"` with a 400 error. Even when `supported` is `true`, the API can still reject a `"between_tools"` request for another reason, such as an effort level that the model doesn't allow with

`"between_tools"`. For Claude Sonnet 5.5, for example, the Models API reports `thinking.types.between_tools.supported` as `true` and `thinking.types.disabled.supported` as `false`.

Each model's `capabilities` object also includes `server_tools`, which reports whether the model accepts the web search and code execution tools. It doesn't cover other server tools, such as web fetch. `server_tools.web_search.supported` and `server_tools.code_execution.supported` are `true` when the model accepts at least one version of that tool, not necessarily every version. `server_tools.supported` is `true` when the model accepts at least one of the two tools. Even when a tool is supported, your organization's settings can still cause a request that uses it to fail. For example, an administrator can disable web search.

The top-level `code_execution` capability is a different check. It reports whether code that Claude runs in the code execution tool can call the request's other tools, as in programmatic tool calling. For Claude Haiku 4.5, for example, the Models API reports `server_tools.code_execution.supported` as `true` and `code_execution.supported` as `false`.

The API reference lists the List Models endpoint's parameters and full response.

List Models

↙ GET `/v1/models`

List available models.

Query parameters `after_id` `before_id` `lifecycle` `limit`

`after_id`: optional string
ID of the object to use as a cursor for pagination. When provided, returns the page of results immediately after this object.

`before_id`: optional string
ID of the object to use as a cursor for pagination. When provided, returns the page of results immediately before this object.

`lifecycle`: optional array of `"active"` or `"deprecated"` or `"retired"`
Filter the list to models in any of the given lifecycle stages (`active`, `deprecated`, or `retired`). Up to 3 values. When omitted, the list contains the `active` and `deprecated` models; `retired` models appear only when `retired` is requested explicitly.

maxItems: 3

One of the following:

```
"active"
"deprecated"
"retired"
```

limit: optional number
Number of items to return per page.

Defaults to `20`. Ranges from `1` to `1000`.

default: `20`, minimum: `1`, maximum: `1000`

Headers `anthropic-version` `anthropic-workspace-id`

"anthropic-version": optional string
The version of the Claude API you want to use.

Read more about versioning and our version history here.

"anthropic-workspace-id": optional string
Optional header to select the Workspace for this request. The value is a Workspace ID (for example, `wrkspc_011CZkZaBF1tNoB5wlCeusgy`).

Only needed for credentials that can act on more than one Workspace. A credential that belongs to a specific Workspace may omit it; if sent, it must match that Workspace.

Returns `data` `first_id` `has_more` `last_id`

data: array of **ModelInfo**

type: `"model"`
Object type.

For Models, this is always `"model"`.

default: `model`

id: string
Unique model identifier.

capabilities: **ModelCapabilities** or null
Object mapping capability names to their support details. Keys are always present for all known capabilities.

batch: **CapabilitySupport**
Whether the model supports the Batch API.

supported: boolean
Whether this capability is supported by the model.

citations: **CapabilitySupport**
Whether the model supports citation generation.

supported: boolean
Whether this capability is supported by the model.

code_execution: **CapabilitySupport**
Whether code that the model runs in the code execution tool can call the request's other tools, as in programmatic tool calling and dynamic filtering for web search and web fetch. Support for the code execution tool itself is in `server_tools.code_execution`.

supported: boolean
Whether this capability is supported by the model.

context_management: **ContextManagementCapability**
Context management support and available strategies.

clear_thinking_20251015: **CapabilitySupport** or null
Whether the clear_thinking_20251015 strategy is supported.

supported: boolean
Whether this capability is supported by the model.

clear_tool_uses_20250919: **CapabilitySupport** or null
Whether the clear_tool_uses_20250919 strategy is supported.

supported: boolean
Whether this capability is supported by the model.

compact_20260112: **CapabilitySupport** or null
Whether the compact_20260112 strategy is supported.

supported: boolean
Whether this capability is supported by the model.

supported: boolean
Whether this capability is supported by the model.

effort: **EffortCapability**
Effort (reasoning_effort) support and available levels.

high: **CapabilitySupport**
Whether the model supports high effort level.

supported: boolean
Whether this capability is supported by the model.

low: **CapabilitySupport**
Whether the model supports low effort level.

> **supported**: boolean
> Whether this capability is supported by the model.

max: **CapabilitySupport**
Whether the model supports max effort level.

> **supported**: boolean
> Whether this capability is supported by the model.

medium: **CapabilitySupport**
Whether the model supports medium effort level.

> **supported**: boolean
> Whether this capability is supported by the model.

supported: boolean
Whether this capability is supported by the model.

xhigh: **CapabilitySupport** or null
Whether the model supports xhigh effort level.

> **supported**: boolean
> Whether this capability is supported by the model.

image_input: **CapabilitySupport**
Whether the model accepts image content blocks.

> **supported**: boolean
> Whether this capability is supported by the model.

pdf_input: **CapabilitySupport**
Whether the model accepts PDF content blocks.

> **supported**: boolean
> Whether this capability is supported by the model.

server_tools: **ServerToolsCapability**
Whether this model supports the web search and code execution server tools. `supported` is true when the model supports at least one of the tools. A supported tool can still be rejected for your organization, for example when an admin has turned web search off.

code_execution: **CapabilitySupport**
Whether the model supports the code execution tool: true when the model supports at least one version of the tool, not necessarily every version.

> **supported**: boolean
> Whether this capability is supported by the model.

supported: boolean
Whether this capability is supported by the model.

web_search: **CapabilitySupport**
Whether the model supports the web search tool: true when the model supports at least one version of the tool, not necessarily every version.

> **supported**: boolean
> Whether this capability is supported by the model.

structured_outputs: **CapabilitySupport**
Whether the model supports structured output / JSON mode / strict tool schemas.

> **supported**: boolean
> Whether this capability is supported by the model.

thinking: **ThinkingCapability**
Thinking capability and supported type configurations.

> **supported**: boolean
> Whether this capability is supported by the model.

types: **ThinkingTypes**
Supported thinking type configurations.

adaptive: **CapabilitySupport**
Whether the model accepts thinking with type 'adaptive' (the model decides whether and how much to think).

> **supported**: boolean
> Whether this capability is supported by the model.

between_tools: **CapabilitySupport**
Whether the model accepts thinking with type 'between_tools' (thinking turned off, and any short progress updates the model writes between tool calls come back as thinking blocks). False exactly when a request that sends it gets a 400 from this model. A request that sends 'between_tools' can still be rejected for another reason, such as an effort level the model does not allow with thinking off.

> **supported**: boolean
> Whether this capability is supported by the model.

disabled: **CapabilitySupport**
Whether the model accepts thinking with type 'disabled' (thinking turned off). False exactly when a request that sends it gets a 400 from this model. True on a model that does not support thinking. A request that sends 'disabled' can still be rejected for another reason, such as an effort level the model does not allow with thinking off.

> **supported**: boolean
> Whether this capability is supported by the model.

enabled: **CapabilitySupport**
Whether the model accepts thinking with type 'enabled' (extended thinking with a caller-set `budget_tokens`).

> **supported**: boolean
> Whether this capability is supported by the model.

created_at: string
RFC 3339 datetime string representing the time at which the model was released. May be set to an epoch value if the release date is unknown.

format: `date-time`

deprecated_at: string or null
RFC 3339 datetime string representing the time of the model's most recent deprecation. Populated for `deprecated` and `retired` models; `null` while the model is `active`.

format: `date-time`

display_name: string
A human-readable name for the model.

lifecycle: `"active"` or `"deprecated"` or `"retired"`
The model's current lifecycle stage.

- `active`: The model is available for use, open to new adopters, and not scheduled for retirement.

- `deprecated`: The model remains callable for organizations with existing access, but is headed for retirement and closed to new adopters.
- `retired`: The model is no longer available for use; inference requests naming it fail. It remains in the catalogue as the historical record of its retirement.

default: `active`

One of the following:

```
"active"
"deprecated"
"retired"
```

`line`: **`ModelLine`** or null
The model line this model belongs to, such as `opus` for both Claude Opus 4.5 and Claude Opus 4.6. More lines may be added. `null` when the model belongs to no line; do not infer a line from the `id`.

One of the following:

```
"haiku"
"sonnet"
"opus"
"fable"
"mythos"
```

`max_input_tokens`: number or null
Maximum input context window size in tokens for this model.

`max_tokens`: number or null
Maximum value for the `max_tokens` parameter when using this model.

`retires_at`: string or null
RFC 3339 datetime string representing the model's currently scheduled retirement date. The schedule can be revised until retirement occurs; `null` while the model is `active` or while no retirement is scheduled. A past date on a `deprecated` model means retirement is overdue, not that it has occurred: `lifecycle` is the retirement signal.

format: `date-time`

`first_id`: string or null
First ID in the `data` list. Can be used as the `before_id` for the previous page.

`has_more`: boolean
Indicates if there are more results in the requested page direction.

`last_id`: string or null
Last ID in the `data` list. Can be used as the `after_id` for the next page.

Prompt and output performance

Current Claude models excel in:

- **Performance:** Top-tier results in reasoning, coding, multilingual tasks, long-context handling, honesty, and image processing. See Prompting best practices for general and model-specific prompting guidance.
- **Engaging responses:** Claude models are ideal for applications that require rich, human-like interactions. If you prefer more concise responses, adjust your prompts to guide the model toward the output length you want. Refer to the prompt engineering guides for details.
- **Output quality:** When migrating from a previous model generation, you might notice larger improvements in overall performance. If you're on Claude Opus 5 or earlier, see the Claude Opus 5.5 migration guide.

Get started with Claude

Whether you're a developer looking to integrate Claude into your applications or a user wanting to experience the power of AI firsthand, the following resources can help.



Intro to Claude

Explore Claude's capabilities and development flow.



Quickstart

Learn how to make your first API call in minutes.



Choosing a model

Establish criteria and pick the right model for your use case.



Pricing

Complete pricing, including batch discounts and prompt caching rates.



Model deprecations

Lifecycle status and retirement commitments for every model.



Claude Console

Craft and test prompts directly in your browser.

Looking to chat with Claude? Visit claude.ai. If you have questions, contact the support team or the Discord community.